PEDRO IGOR CAMPOS COSTA

PROJETO LASTRO

Prova de conceito para entrega orientada por especificações, evidências de qualidade e rastreabilidade de releases

Salvador
2026

PEDRO IGOR CAMPOS COSTA

PROJETO LASTRO

Prova de conceito apresentada como documentação técnica de portfólio, com o objetivo de demonstrar decisões arquiteturais, práticas de qualidade e integração entre Java, Spring Boot, Angular, SDD e rastreabilidade de entregas.

Salvador
2026

RESUMO

Esta prova de conceito apresenta o Projeto Lastro, uma aplicação para registrar mudanças de software como especificações revisáveis e vincular critérios de aceite, evidências de qualidade e decisões de release. O sistema combina Java 21, Spring Boot 4, PostgreSQL e Angular 21. A arquitetura mantém as regras de negócio fora dos frameworks, oferece uma API REST para o fluxo operacional e publica consultas MCP somente leitura para integrações. A validação incluiu testes automatizados, regra arquitetural, análise estática, cobertura de 97,22% na lógica de negócio, build de produção e execução dos três serviços em containers. O resultado não pretende substituir Jira, GitLab ou Jenkins. Seu papel é preservar o vínculo entre a intenção da mudança e a evidência que sustenta a decisão de publicar.

Palavras-chave: Java. Spring Boot. Angular. Spec-Driven Development. MCP. Qualidade de software.

ABSTRACT

This proof of concept presents the Projeto Lastro platform, an application that records software changes as reviewable specifications and connects acceptance criteria, quality evidence and release decisions. The solution combines Java 21, Spring Boot 4, PostgreSQL and Angular 21. Business rules remain independent from frameworks, while REST supports the operational workflow and read-only MCP tools provide context to integration clients. Verification covered automated tests, an architecture rule, static analysis, 97.22% business-logic coverage, a production build and container execution. Lastro complements delivery tools by preserving the link between a change intention and the evidence used to authorize its release.

Keywords: Java. Spring Boot. Angular. Spec-Driven Development. MCP. Software quality.

SUMÁRIO

1 INTRODUÇÃO.....5
2 PROBLEMA E OBJETIVOS.....5
3 FUNDAMENTAÇÃO TÉCNICA.....6
4 ARQUITETURA DA SOLUÇÃO.....6
5 IMPLEMENTAÇÃO.....7
6 SEGURANÇA E CONFIABILIDADE.....10
7 ESTRATÉGIA DE TESTES E QUALIDADE.....10
8 PLANEJAMENTO E REVISÃO.....10
9 EXECUÇÃO DA PROVA DE CONCEITO.....11
10 DECISÕES, LIMITES E EVOLUÇÃO.....11
11 CONCLUSÃO.....12
REFERÊNCIAS.....12

1 INTRODUÇÃO

Uma entrega pode passar pelo pipeline e ainda assim chegar ao ambiente sem uma explicação clara sobre o comportamento validado. Esse problema aparece quando requisito, decisão técnica, teste e aprovação vivem em ferramentas diferentes. O Projeto Lastro foi construído para experimentar outro ponto de partida: a especificação é o contrato de engenharia, e a prontidão de release é uma conclusão derivada de evidências, não uma caixa marcada manualmente.

A aplicação foi planejada a partir de uma especificação versionada antes do código. As decisões que afetam estrutura, planejamento e interfaces de integração foram registradas em Architecture Decision Records (ADRs). Essa sequência permite examinar não apenas o resultado, mas também os critérios e os trade-offs adotados.

2 PROBLEMA E OBJETIVOS

2.1 Problema

Times de produto usam Jira para coordenação, repositórios para código, SonarQube para análise e pipelines para implantação. Essas ferramentas são úteis, mas não garantem sozinhas que cada critério da mudança tenha uma evidência identificável. A fragmentação aumenta o custo da revisão e permite que um release pareça pronto mesmo quando restam comportamentos sem validação.

2.2 Objetivo geral

Construir e validar uma aplicação full stack que preserve a rastreabilidade entre problema, solução proposta, critérios de aceite, revisão técnica, implementação e release.

2.3 Objetivos específicos

- modelar transições explícitas e impedir estados inválidos no domínio;
- apresentar bloqueadores concretos quando a entrega não está pronta;
- expor uma interface Angular utilizável sem duplicar regras de negócio;
- oferecer contexto para integrações por meio de consultas MCP somente leitura;
- produzir evidências automatizadas de testes, cobertura, análise estática e composição;

- documentar riscos, decisões arquiteturais, operação e possibilidades de evolução.

3 FUNDAMENTAÇÃO TÉCNICA

3.1 Spec-Driven Development

No desenvolvimento orientado por especificações, o comportamento esperado é escrito antes da implementação e permanece versionado com ela. No Lastro, cada critério recebe identidade e pode ser verificado de forma independente. O planner produz tarefas que citam esses identificadores, o que reduz a distância entre planejamento e teste.

3.2 Arquitetura hexagonal

O domínio não importa Spring, JPA ou classes de transporte. Serviços de aplicação coordenam casos de uso, enquanto repositório e planner são portas. Essa separação torna as regras executáveis em testes rápidos e permite trocar adaptadores sem reescrever o fluxo de negócio.

3.3 Interfaces de consulta

Model Context Protocol oferece uma fronteira padronizada para ferramentas de consulta. A POC publica listagem, detalhe e prontidão. Nenhuma ferramenta altera o sistema. Essa restrição reduz o impacto de um prompt incorreto e mantém decisões de fluxo nas APIs controladas.

4 ARQUITETURA DA SOLUÇÃO

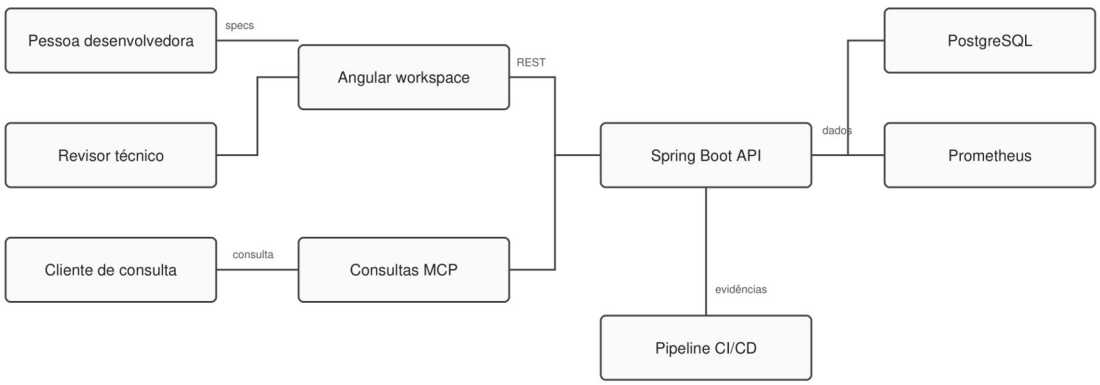


Figura 1 — Contexto do Projeto Lastro

Fonte: elaboração própria (2026).

A pessoa desenvolvedora e o revisor usam o painel Angular. A interface chama a API com uma chave de desenvolvimento; a API persiste o agregado em PostgreSQL. Pipelines registram referências de evidência, enquanto clientes de integração consultam o mesmo estado por MCP. O Actuator publica saúde e métricas.

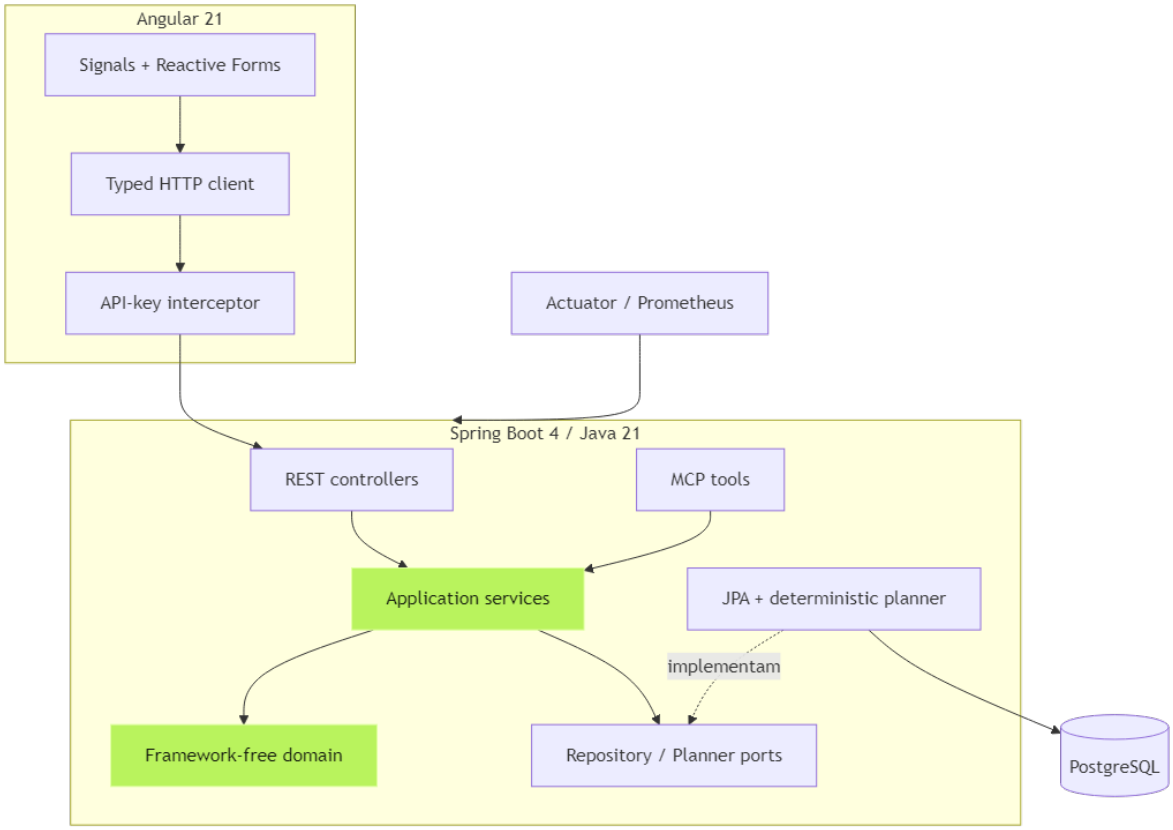


Figura 2 — Componentes e dependências

Fonte: elaboração própria (2026).

As setas confirmam a direção das dependências: adaptadores conhecem portas, mas o domínio não conhece infraestrutura. Um teste com ArchUnit impede regressões nessa fronteira.

5 IMPLEMENTAÇÃO

5.1 Modelo de domínio

ChangeSpec é o agregado responsável pelas transições e pela avaliação de prontidão. Uma spec nasce em DRAFT, passa por revisão, aprovação e implementação. Retorno para rascunho exige motivo; aprovação exige comentário;

gate aprovado exige referência; release exige todos os critérios verificados e os cinco gates aprovados.

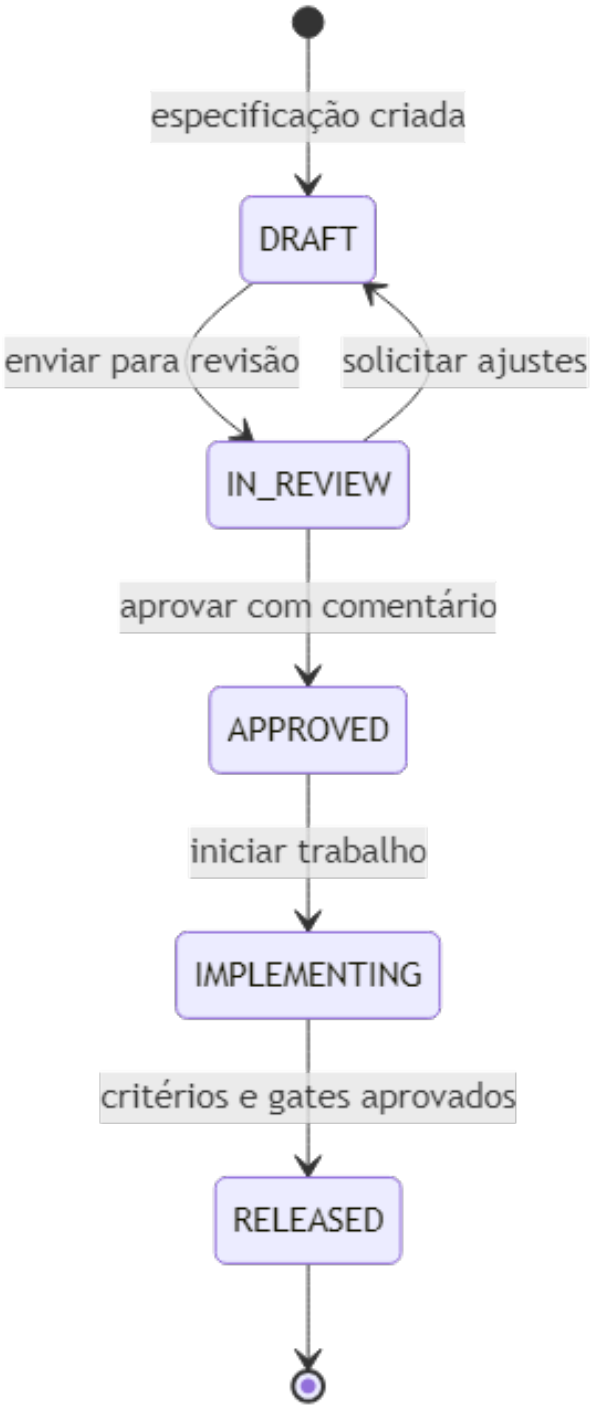


Figura 3 — Ciclo de vida da especificação
Fonte: elaboração própria (2026).

5.2 Backend

O backend usa Java 21 e Spring Boot 4. A API aplica Bean Validation e responde falhas de negócio com Problem Details. Flyway cria o esquema, JPA implementa a persistência e o campo de versão oferece lock otimista. A configuração desativa Open Session in View para manter o carregamento de dados dentro de transações explícitas.

5.3 Frontend

O painel Angular 21 usa componentes standalone, signals para estado derivado, Reactive Forms e HttpClient tipado. O navegador mostra as ações permitidas pelo status, mas o backend valida cada transição novamente. Essa escolha trata a interface como cliente não confiável.

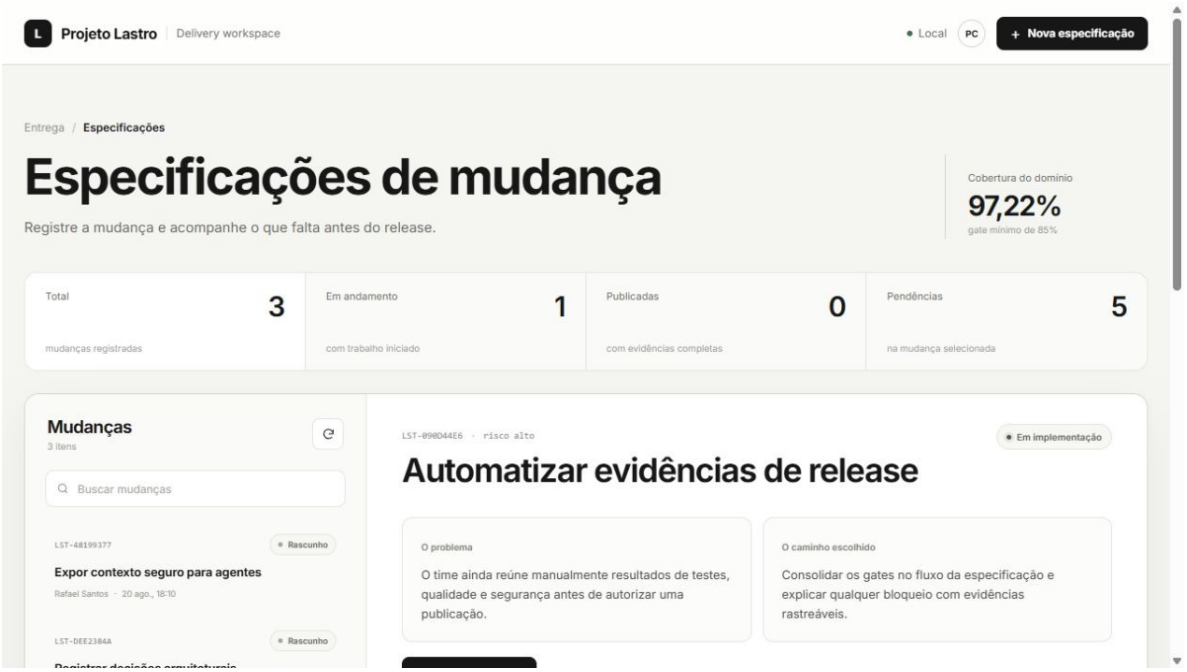


Figura 4 — Painel Angular executado com dados de demonstração

Fonte: elaboração própria (2026).

6 SEGURANÇA E CONFIABILIDADE

A POC usa chave de API comparada em tempo constante, requisições sem sessão e CORS restrito ao cliente local. O container Java executa com usuário sem privilégios. O threat model reconhece que uma chave compartilhada não basta para produção e propõe OIDC, papéis e gestão de segredos.

Risco	Controle atual	Evolução
Atualização concorrente	Lock otimista	Resposta de conflito com versão

Risco	Controle atual	Evolução
		atual
Evidência forjada	Referência obrigatória e auditoria	Assinatura do provedor de CI
Cliente com excesso de permissão	MCP somente leitura	Revisão formal de novas consultas
Dependência comprometida	Lockfile, SBOM e Dependabot	Assinatura de imagens

7 ESTRATÉGIA DE TESTES E QUALIDADE

O comando Maven verify executa testes de domínio, serviços, contexto Spring e arquitetura, além de Checkstyle, SpotBugs, JaCoCo e geração de SBOM. Foram executados 17 testes sem falhas. A área de domínio e serviços atingiu 97,22% de cobertura de linhas, acima do gate de 85%. DTOs, configuração e adaptadores não entram nesse indicador; o recorte está explícito no build.

Evidência	Resultado observado
Testes backend	17 aprovados; nenhuma falha ou salto
Cobertura de negócio	97,22%; gate mínimo de 85%
Checkstyle	0 violações
SpotBugs	0 achados
Testes Angular	2 aprovados
Bundle Angular	251,68 KB bruto; 65,39 KB estimados na transferência
Containers	PostgreSQL, backend e frontend saudáveis

8 PLANEJAMENTO E REVISÃO

O processo de contribuição exige leitura da especificação, vínculo com critérios, consulta aos ADRs e execução das verificações antes de concluir uma tarefa. A revisão relaciona cada mudança às evidências de comportamento, segurança e qualidade necessárias para a entrega.

O planner local é deliberadamente determinístico. Ele cria tarefas para backend, frontend, testes e documentação e referencia todos os critérios. Outros adaptadores podem implementar a mesma porta, preservando os contratos e os testes do caso de uso.

9 EXECUÇÃO DA PROVA DE CONCEITO

A validação seguiu quatro etapas: verificação isolada do backend; build e teste do Angular; build das imagens; inicialização do Compose com health checks. Depois, três specs de demonstração foram criadas pela própria API. A spec principal avançou até IMPLEMENTING e permaneceu bloqueada por três critérios de aceite e cinco gates ainda sem evidência.

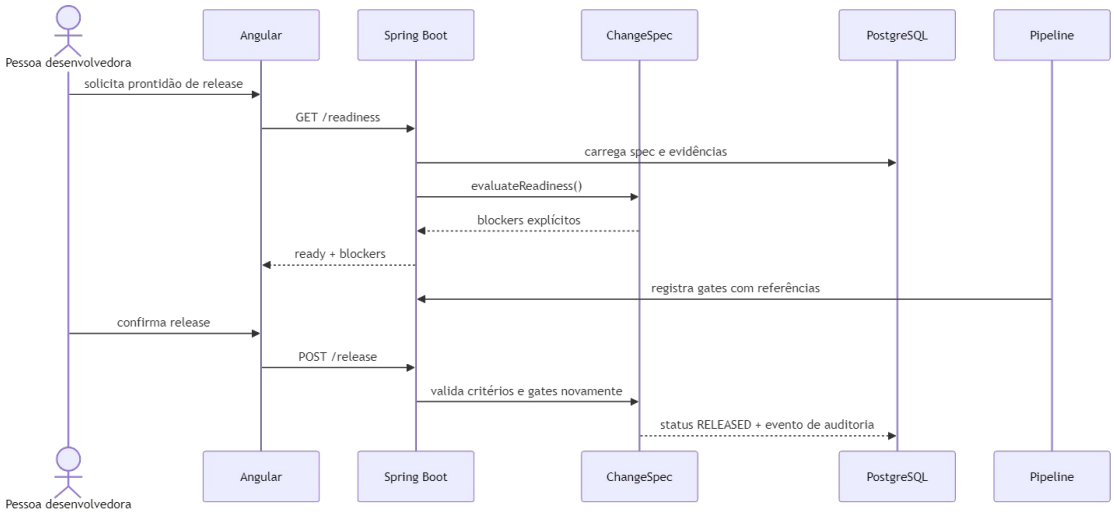


Figura 5 — Avaliação e confirmação de release

Fonte: elaboração própria (2026).

O cenário confirma uma decisão importante: o botão de release pode aparecer durante a implementação, mas a tentativa é recusada enquanto evaluateReadiness encontrar bloqueadores. A regra permanece atômica no servidor.

10 DECISÕES, LIMITES E EVOLUÇÃO

Decisão	Benefício	Custo assumido
Monorepo	Uma mudança preserva código, spec e evidência	Pipelines precisam separar caches
Domínio sem framework	Testes rápidos e regras explícitas	Mapeamento JPA adicional
Planner local	Determinismo, privacidade e custo zero	Menor adaptação semântica
MCP somente leitura	Menor superfície de risco	Clientes não automatizam transições
API key na POC	Execução local simples	Inadequada para identidades corporativas

As próximas evoluções recomendadas são OIDC, verificação assinada de evidências de CI, paginação, integração real com Jira, testes end-to-end e observabilidade distribuída. Novos adaptadores devem preservar os critérios de aceite e ser avaliados com cenários reproduzíveis.

11 CONCLUSÃO

O Projeto Lastro reúne SDD, qualidade e rastreabilidade em uma trilha de entrega verificável. A principal entrega não é um CRUD de especificações, mas um conjunto de invariantes: a mudança precisa ser revisada, cada critério precisa de verificação e cada gate precisa de evidência. Os testes, a análise estática e a

execução em containers confirmam a viabilidade técnica da POC. Os limites registrados indicam com honestidade o trabalho necessário para levar o desenho a um ambiente corporativo.

REFERÊNCIAS

ANGULAR. Angular documentation. Disponível em: <https://angular.dev/>. Acesso em: 20 ago. 2026.

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

GITHUB. GitHub Actions documentation. Disponível em: <https://docs.github.com/actions>. Acesso em: 20 ago. 2026.

MODEL CONTEXT PROTOCOL. Specification. Disponível em: <https://modelcontextprotocol.io/>. Acesso em: 20 ago. 2026.

OWASP FOUNDATION. OWASP Application Security Verification Standard. Disponível em: <https://owasp.org/www-project-application-security-verification-standard/>. Acesso em: 20 ago. 2026.

SPRING. Spring AI MCP Server Boot Starter. Disponível em: <https://docs.spring.io/spring-ai/reference/api/mcp/mcp-server-boot-starter-docs.html>. Acesso em: 20 ago. 2026.

SPRING. Spring Boot reference documentation. Disponível em: <https://docs.spring.io/spring-boot/>. Acesso em: 20 ago. 2026.

VERNON, Vaughn. Implementing Domain-Driven Design. Boston: Addison-Wesley, 2013.